



UNIVERSIDAD DE LOS LAGOS

7

Programación Orientada a Objetos

Docente: Patricio Araneda García

Correo: patricio.aranedagarcia@ulagos.cl



PATRICIO ARANEDA GARCÍA
Magíster en Informática Médica
Universidad de Chile

DATACLINICA Spa



patricioaraneda.cl



dataclinica.cl



/in/patricioaraneda



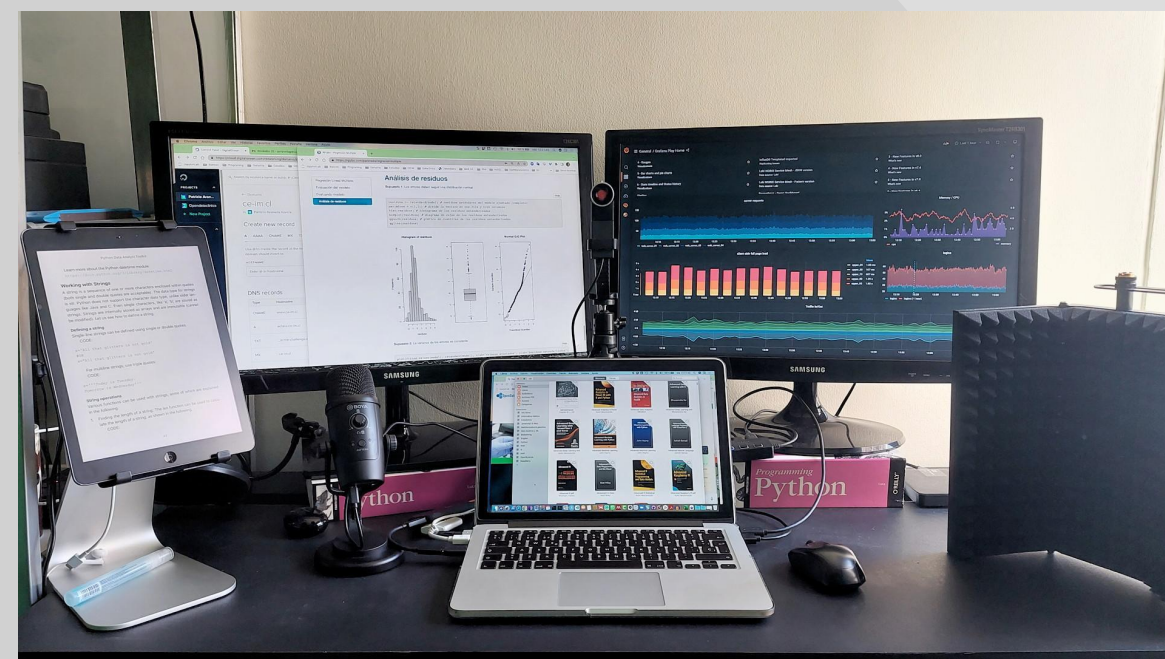
@paranedagarcia

Profesor de Ciencias Naturales y Biología
Pontificia Universidad Católica de Chile

Ingeniero en Computación e Informática
Instituto Profesional Virginio Gómez

Magíster en Informática Médica
Universidad de Chile

- Salud Pública y Redes Asistenciales
- Inteligencia Artificial
- Modelamiento Productivo y Machine Learning



<https://github.com/paranedagarcia>



<https://rpubs.com/paraneda>

DOCENCIA UNIVERSITARIA

- Análisis de Datos
- Bases de Datos
- Fundamentos de Sistemas Inteligentes
- Sistemas y Tecnologías de Información
- Ciencia de Datos y Machine Learning
- Inteligencia Artificial
- Bioestadística



UNIVERSIDAD DE LOS LAGOS

CONTENIDOS DEL CURSO

Manual de Ingeniería Informática

Desarrolla sobre las Tecnologías de la Información

Manual



Fundamentos

Conozca los conceptos básicos de la Ingeniería de la Información y aplique para la resolución de problemas en cualquier ámbito.



Programación

Aprenda las bases de la programación, algoritmos y estructuras de datos, incluyendo la sintaxis, estructuras de control y conceptos fundamentales.



Gobernanza

Aprenda las mejores prácticas para la gobernanza de los datos e información en el ámbito organizacional (en construcción...).

Otros cursos de la serie	Conecta	Más	Licencia
Ingeniería Informática	Website	GitHub	CC BY-NC-SA 4.0 - Solo para uso docente y académico. No olvides citar la fuente y respetar la licencia. Deja una estrella en GitHub si te gustó el curso. ¡Gracias!
Programación en Python	LinkedIn	ORCID	
Bioestadística	Whatsapp	Correo	
Ciencia de Datos			
Análisis con SQL			

<https://patricioaraneda.cl/informatica>

- Programación

Programando con Python

Desde Ciencia de Datos hasta Machine Learning

Curso Python



Programación

Este curso fue diseñado desde cero para ser fácilmente asquible y comprensible para todos. No requiere conocimientos previos de estadística o programación.



Data Science

Se enfoca en la aplicación de la estadística en la investigación médica, y cubre los conceptos y técnicas estadísticas más relevantes para el análisis de datos en este campo.



Desarrollo Continuo

Esta en constante desarrollo, y se irán agregando nuevos contenidos y recursos a medida que se avance en el curso. Se anima a los estudiantes a participar activamente en el curso, compartiendo sus experiencias y sugerencias.

Otros cursos de la serie	Conecta	Más	Licencia
Ingeniería Informática	Website	GitHub	CC BY-NC-SA 4.0 - Solo para uso docente y académico. No olvides citar la fuente y respetar la licencia. Deja una estrella en GitHub si te gustó el curso. ¡Gracias!
Programación en Python	LinkedIn	ORCID	
Bioestadística	Whatsapp	Correo	
Ciencia de Datos			
Análisis con SQL			

<https://patricioaraneda.cl/python/docs/poo/>

"Para investigar la verdad es preciso dudar, en cuanto sea posible, de todas las cosas".

"Si quieres ser un verdadero buscador de la verdad, es necesario que al menos una vez en tu vida dudes, en la medida de lo posible, de todas las cosas".



René Descartes

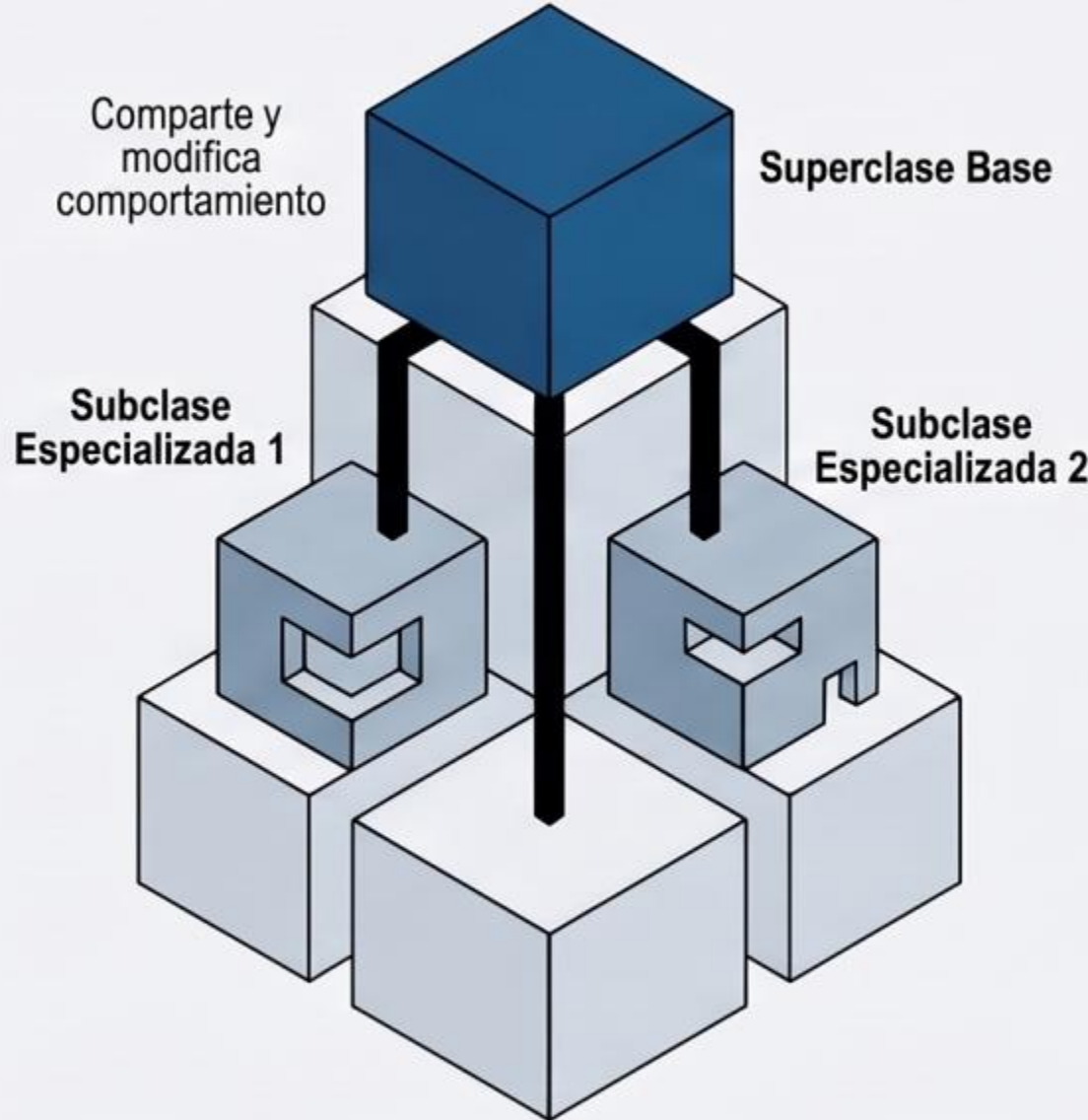
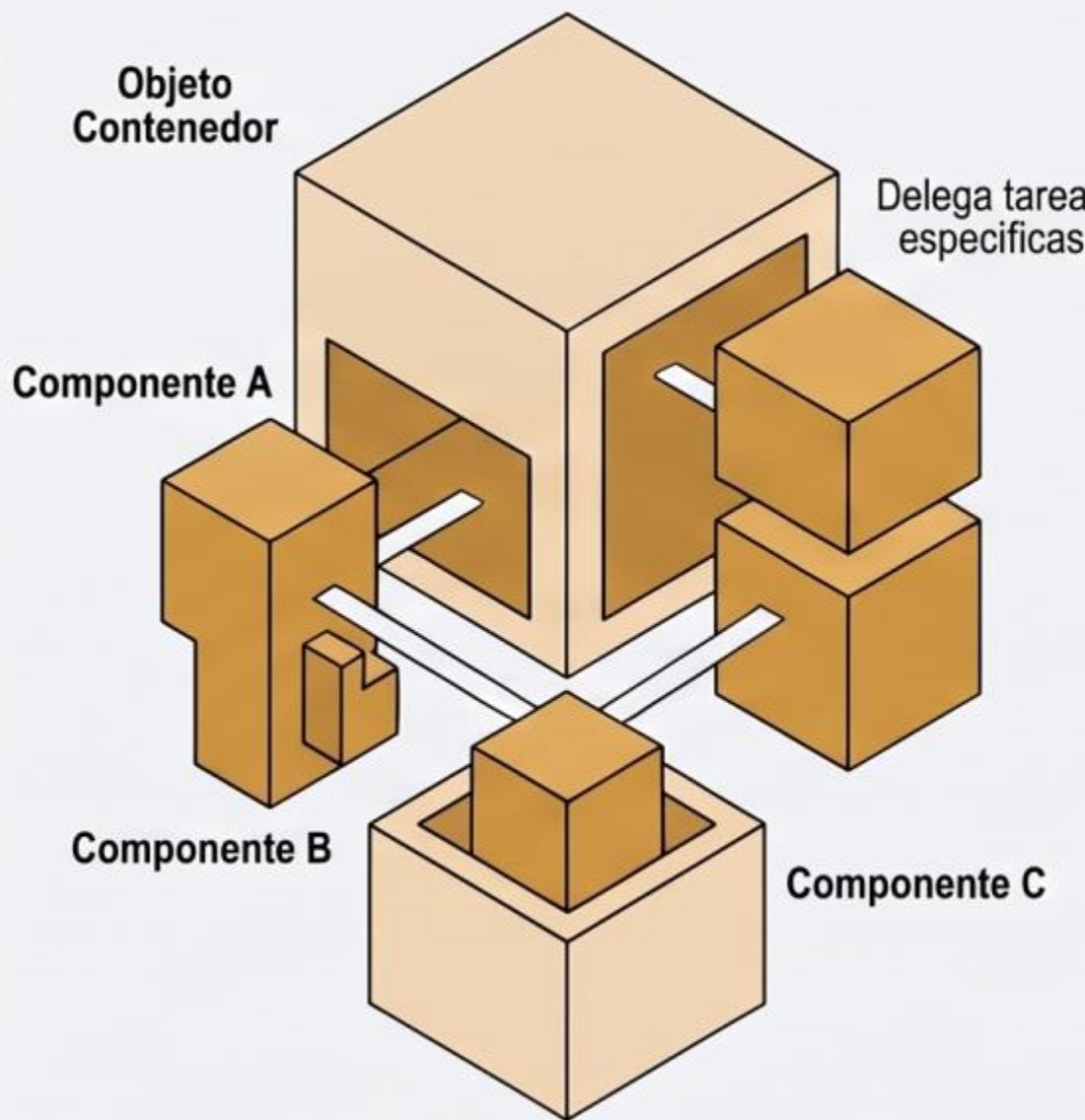
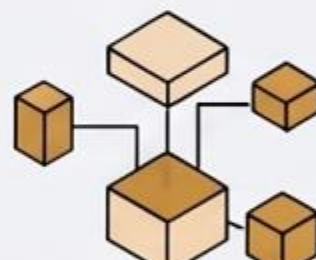
1596 - 1650

The background is a complex digital illustration. It features several server racks with glowing blue lights. In the center, there is a network of green nodes connected by thin, glowing lines. A large, bright orange and yellow sphere is at the center of this network. The background is filled with abstract digital patterns, including binary code (0s and 1s) and various geometric shapes like gears and cubes, all in shades of blue and green.

Composición

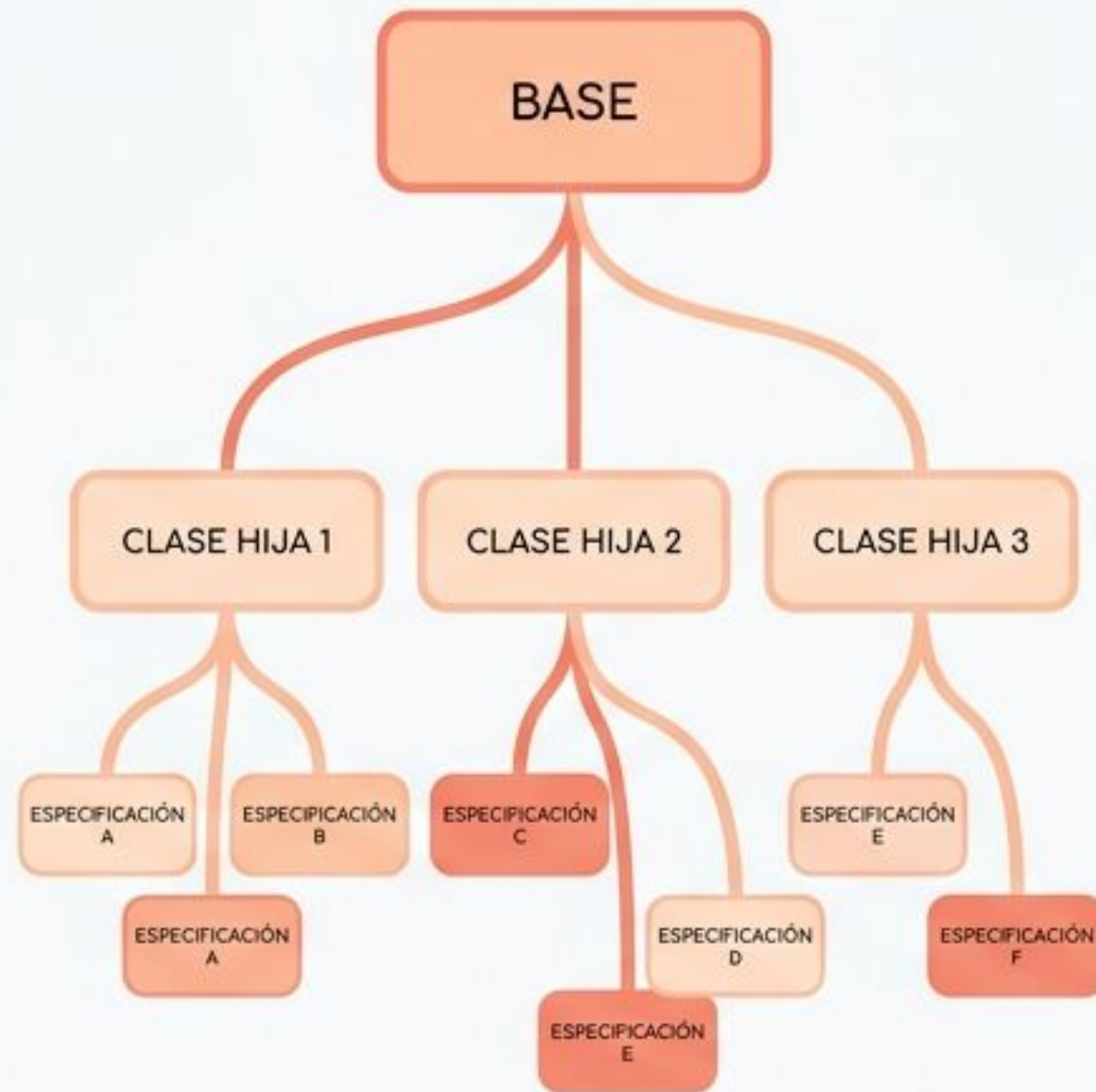
Herencia vs Composición

La Programación Orientada a Objetos ofrece dos vías principales para reutilizar código y modularizar aplicaciones. Mientras que la **Herencia** amplía clases mediante jerarquías directas, la **Composición** ensambla objetos independientes. Elegir entre "Es-Un" y "Tiene-Un" determina la flexibilidad y el mantenimiento del software a largo plazo.

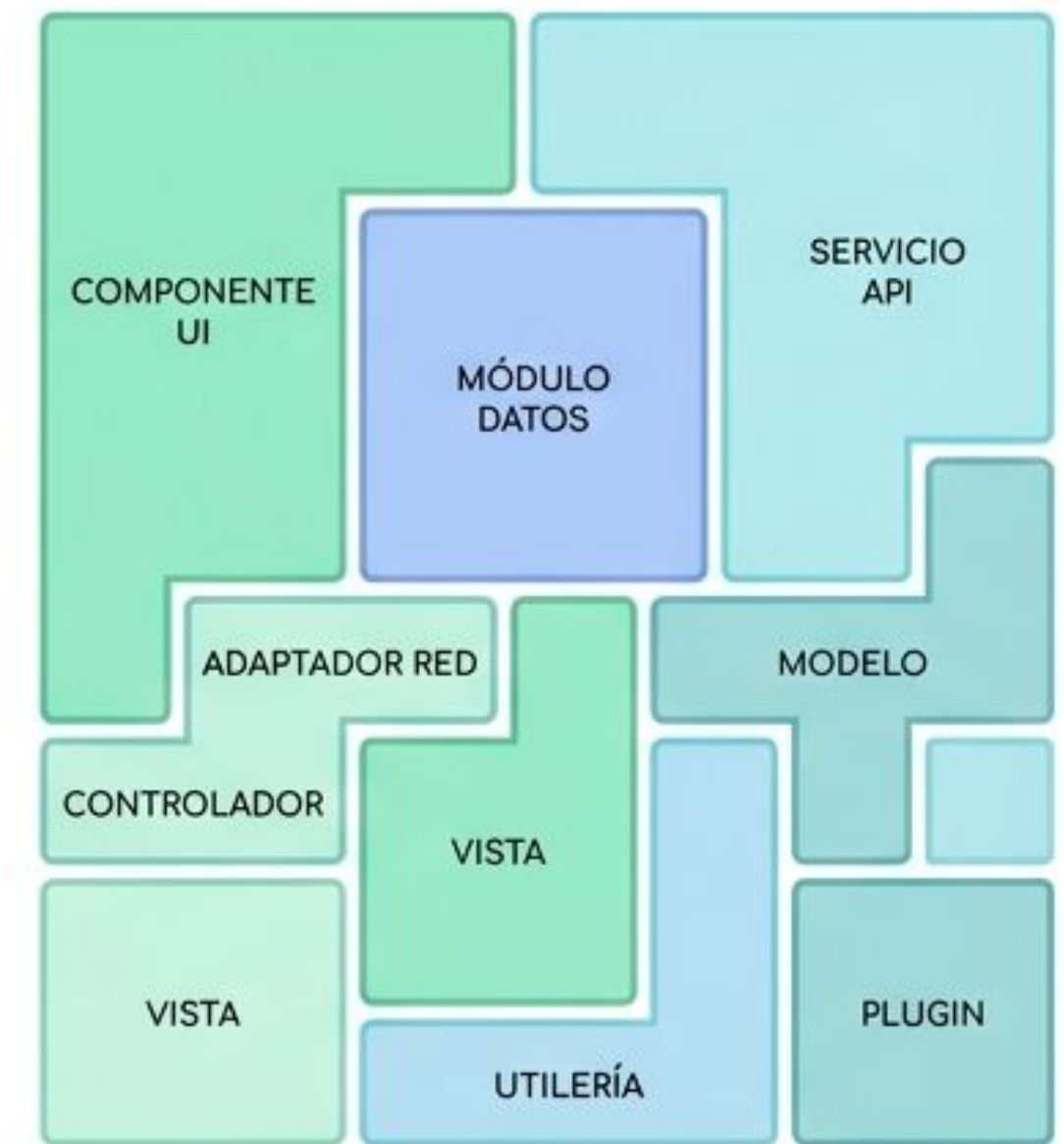
	Herencia ("Es-Un")	Composición ("Tiene-Un")									
 Relación jerárquica estricta: Una subclase especializada deriva de una superclase compartiendo y modificando su comportamiento.	 Comparte y modifica comportamiento Superclase Base Subclase Especializada 1 Subclase Especializada 2	 Objeto Contenedor Delega tareas específicas Componente A Componente B Componente C	 Relación de ensamblaje modular: Un objeto contenedor encapsula otras instancias y les delega tareas específicas.								
 Alto acoplamiento entre clases: Cambiar la superclase afecta directamente a todas las subclases derivadas de ella.			 Bajo acoplamiento y mayor flexibilidad: Permite modificar o cambiar componentes dinámicamente durante la ejecución del programa.								
 Recomendada para taxonomías claras: Ideal cuando la subclase es indiscutiblemente una versión especializada de la base.	 Recomendada para taxonomías claras: Ideal cuando la subclase es indiscutiblemente una versión especializada de la base.		 Recomendada para la reutilización de código: Ideal para combinar múltiples comportamientos independientes sin rigidez jerárquica.								
	<table><tr><th>Herencia</th></tr><tr><td>■ Relación principal: "Es-Un" (ej. Auto es un Vehículo)</td></tr><tr><td>■ Nivel de acoplamiento: Fuerte / Alto</td></tr><tr><td>■ Flexibilidad: Rígida (definida en código)</td></tr></table>	Herencia	■ Relación principal: "Es-Un" (ej. Auto es un Vehículo)	■ Nivel de acoplamiento: Fuerte / Alto	■ Flexibilidad: Rígida (definida en código)	<table><tr><th>Composición</th></tr><tr><td>■ Relación principal: "Tiene-Un" (ej. Auto tiene un Motor)</td></tr><tr><td>■ Nivel de acoplamiento: Débil / Bajo</td></tr><tr><td>■ Flexibilidad: Dinámica (intercambiable en ejecución)</td></tr></table>	Composición	■ Relación principal: "Tiene-Un" (ej. Auto tiene un Motor)	■ Nivel de acoplamiento: Débil / Bajo	■ Flexibilidad: Dinámica (intercambiable en ejecución)	
Herencia											
■ Relación principal: "Es-Un" (ej. Auto es un Vehículo)											
■ Nivel de acoplamiento: Fuerte / Alto											
■ Flexibilidad: Rígida (definida en código)											
Composición											
■ Relación principal: "Tiene-Un" (ej. Auto tiene un Motor)											
■ Nivel de acoplamiento: Débil / Bajo											
■ Flexibilidad: Dinámica (intercambiable en ejecución)											

Herencia vs Composición

Construir software flexible y escalable



Relaciones 'es-un' | Jerarquía rígida | Código compartido por extensión



Relaciones 'tiene-un' | Arquitectura flexible | Código compartido por ensamblaje

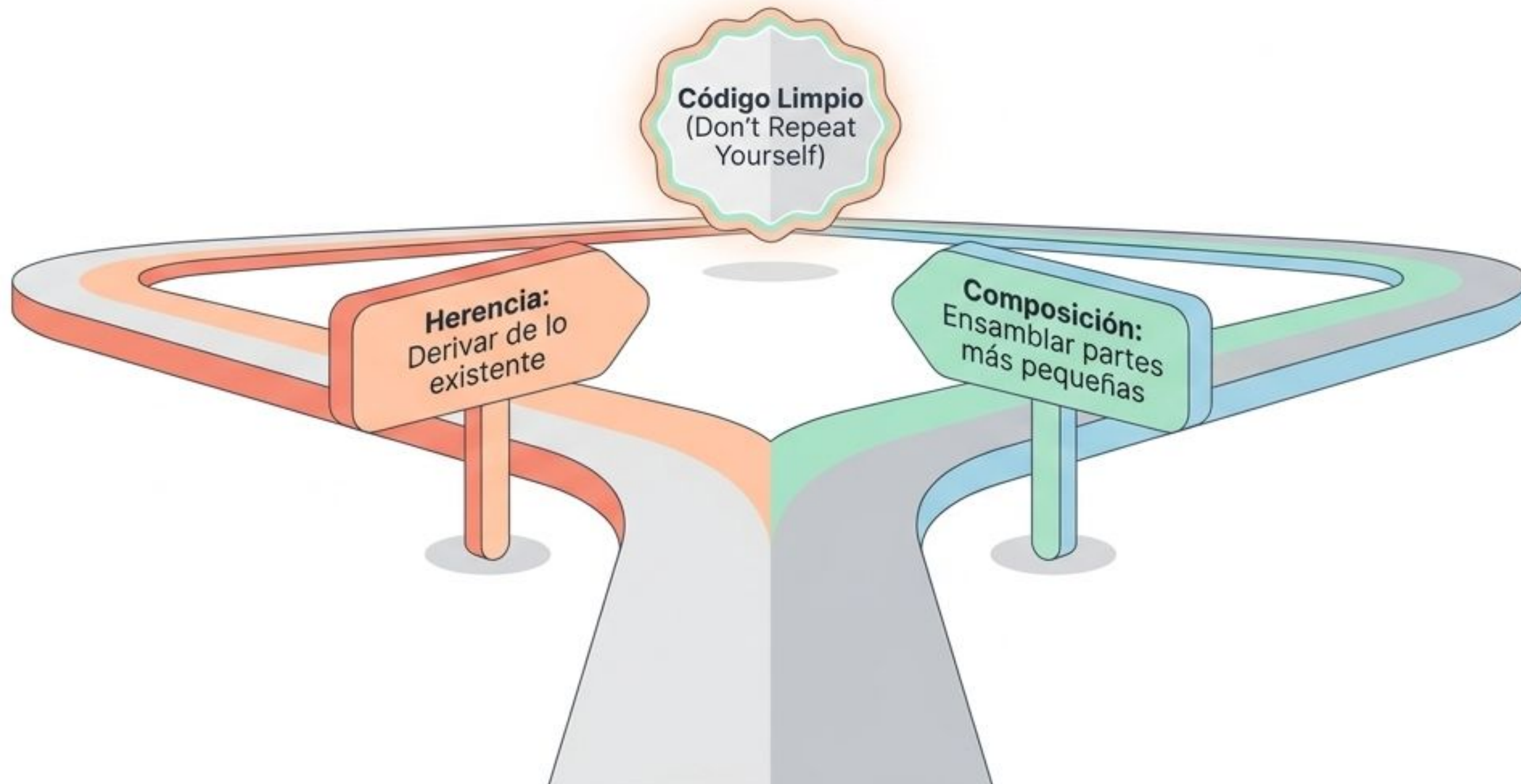
Como se conectan nuestros objetos



En la Programación Orientada a Objetos (POO), no basta con crear clases aisladas. El verdadero reto es cómo estas interactúan para compartir código sin crear un sistema rígido y frágil.

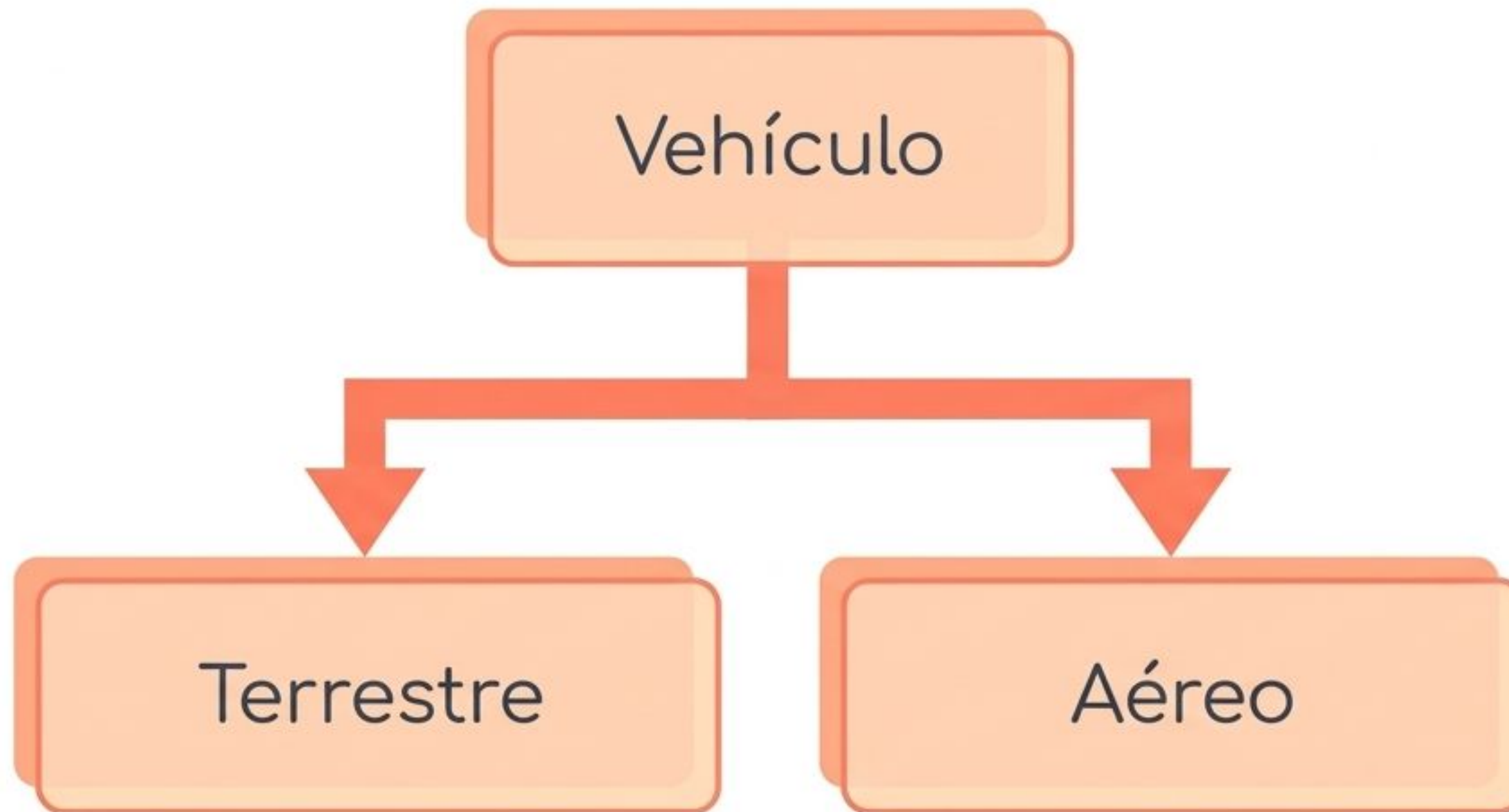
Dos herramientas, un objetivo (DRY)

Para evitar copiar y pegar el mismo código una y otra vez, los arquitectos de software utilizan dos estrategias fundamentales. Ambas buscan lo mismo, pero operan de formas radicalmente distintas.



Herencia: la relación “es-un”

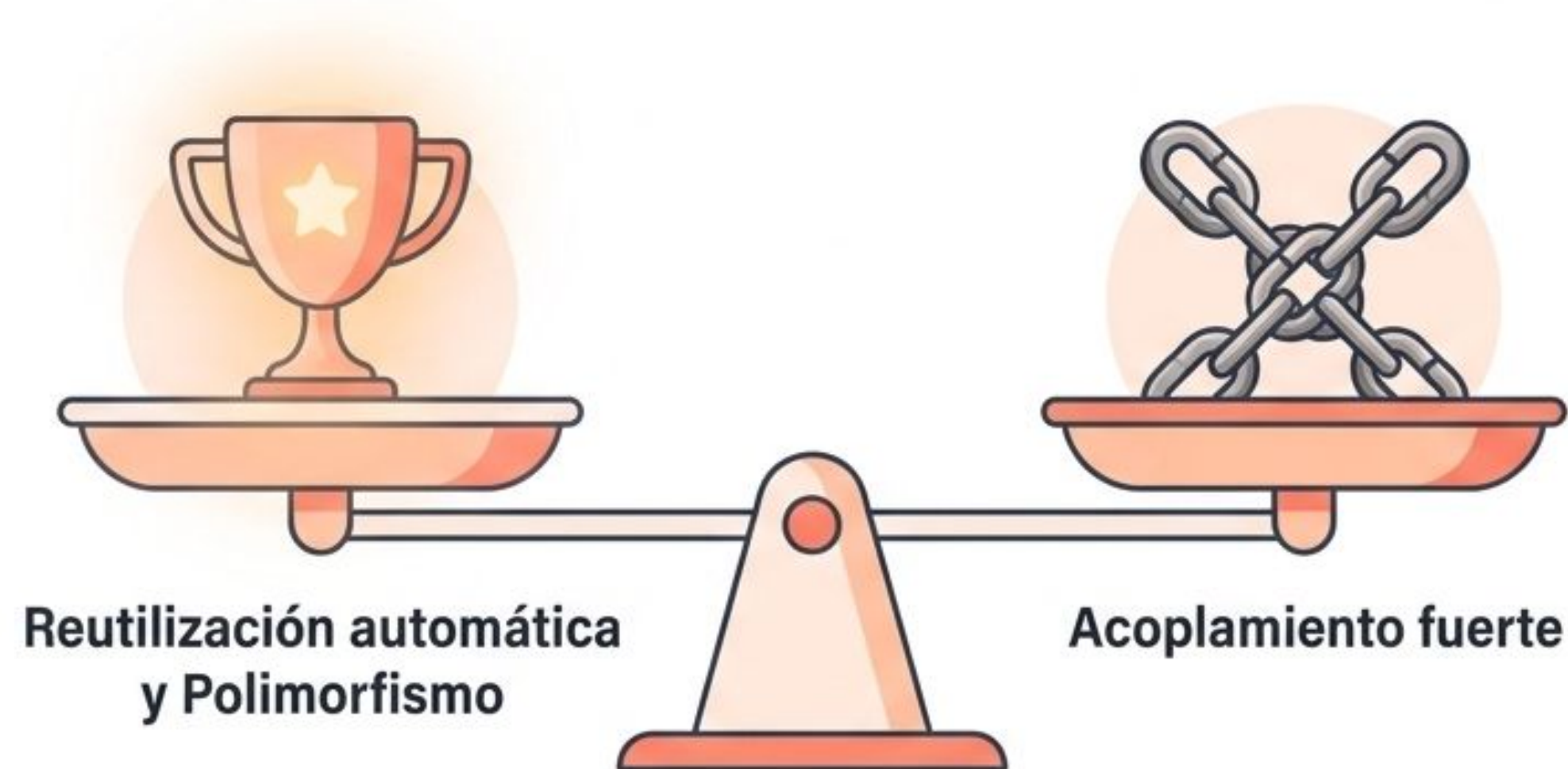
Una clase hija adopta todos los datos y comportamientos de su clase padre. Es una especialización directa.



Regla de oro: Si puedes decir en voz alta "Un Coche es un Vehículo", entonces la herencia tiene sentido.

El poder y peligro de heredar

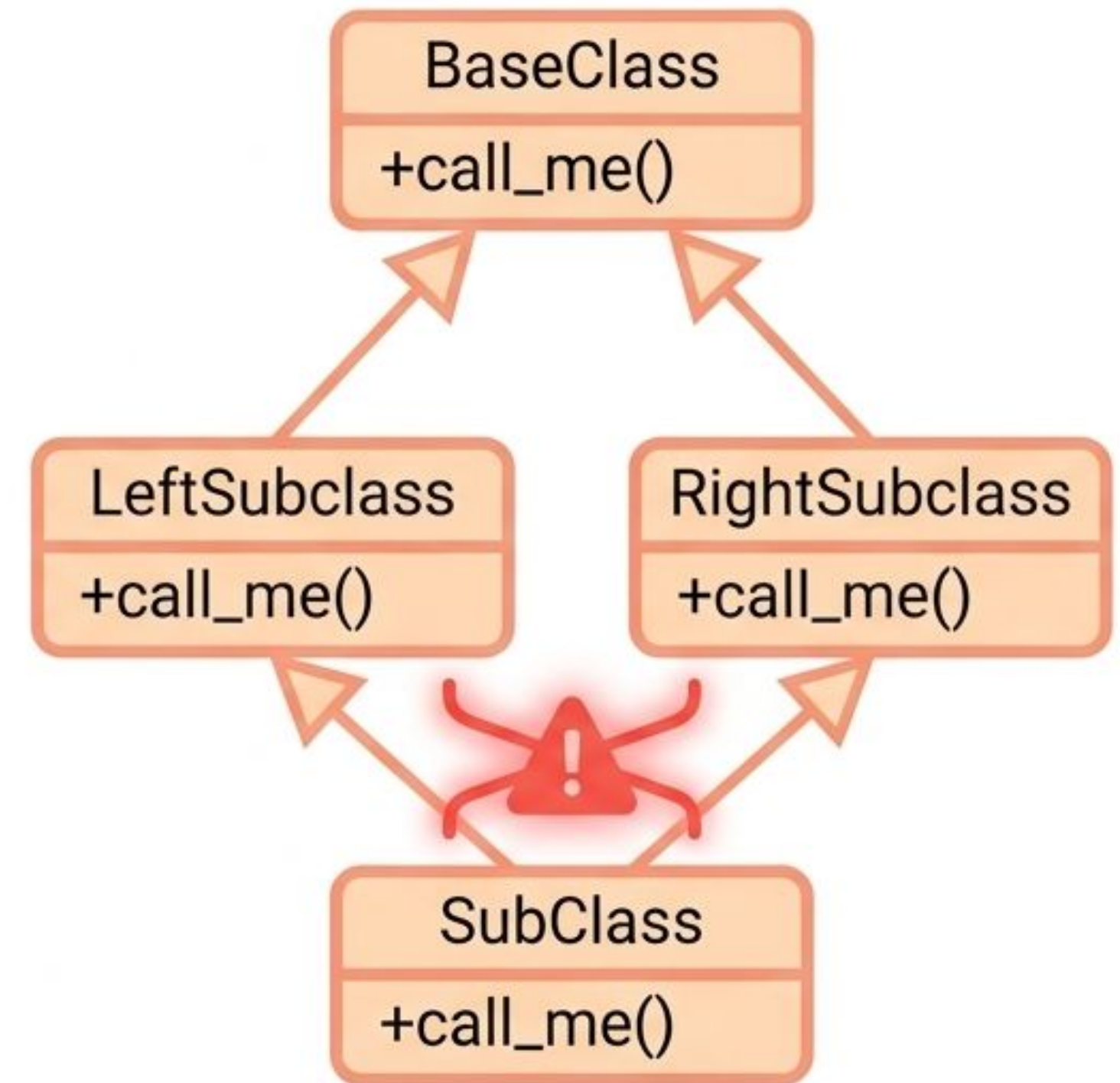
La herencia nos regala código base sin esfuerzo. Sin embargo, ata a las clases hijas de forma permanente. Si cambias un método en el padre, corres el riesgo de romper toda la descendencia.



La trampa de la herencia múltiple

¿Qué pasa si heredas de dos clases que tienen un método con el mismo nombre?

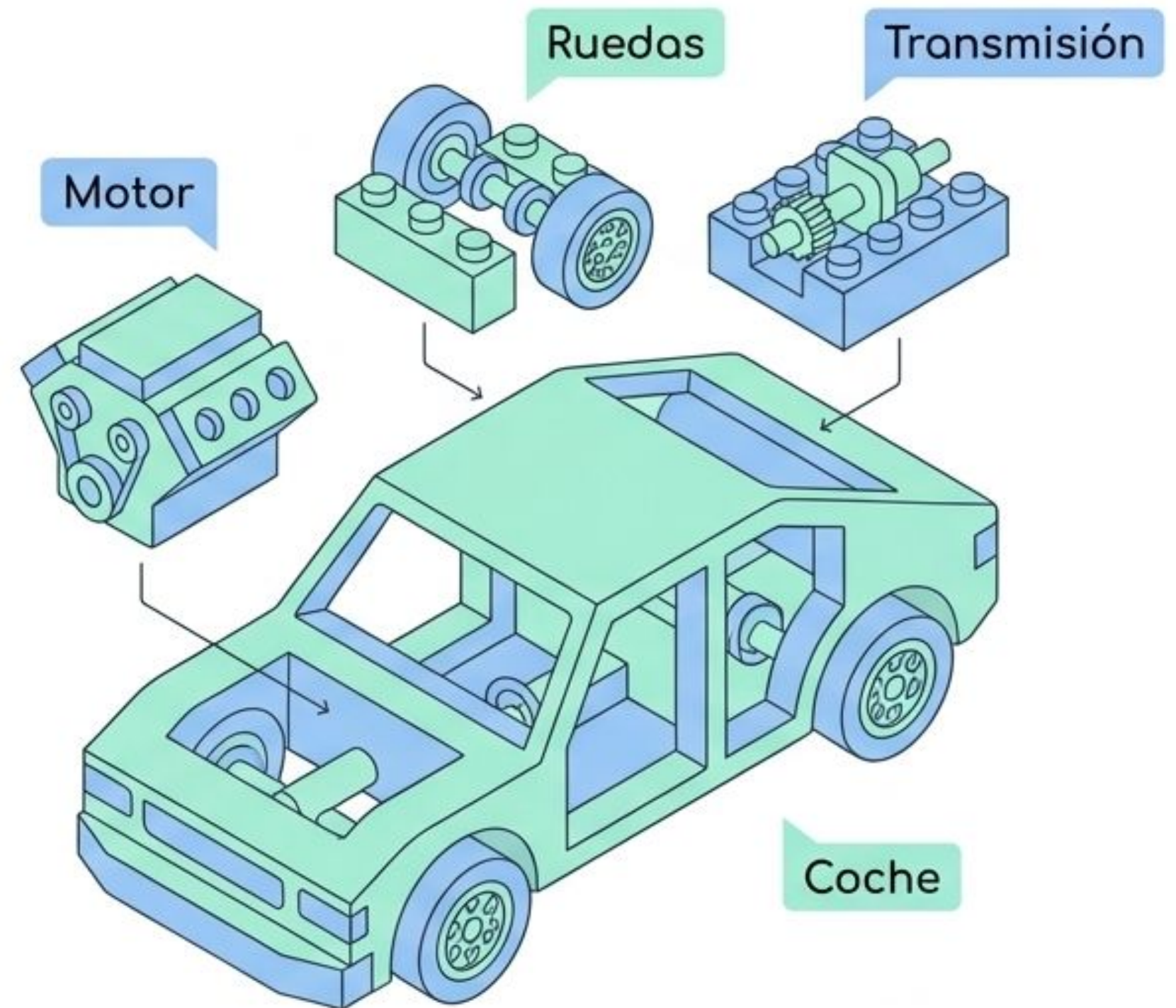
Lenguajes como Python usan el MRO (Method Resolution Order) para adivinar a quién llamar, pero el diseño se vuelve confuso, frágil e impredecible.



Composición: La relación “tiene-un”

En lugar de heredar lo que es, un objeto se construye agrupando lo que tiene.

El objeto principal encapsula instancias de otras clases dentro de su constructor.



Delegación sobre especialización

El objeto compuesto actúa como un director. No necesita saber cómo funciona la combustión interna; simplemente delega el trabajo llamando a los métodos del objeto especializado que contiene.

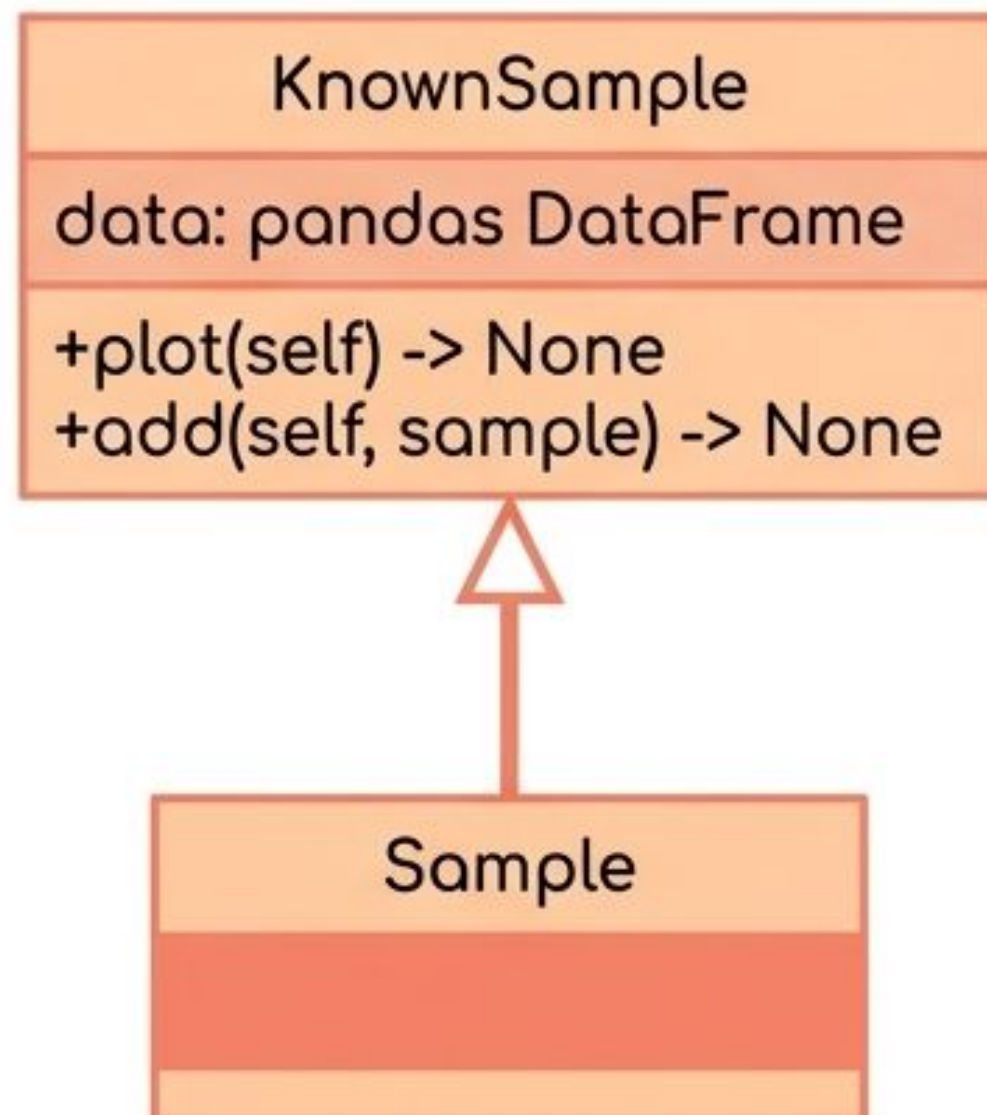


```
...
self.motor.arrancar()
```

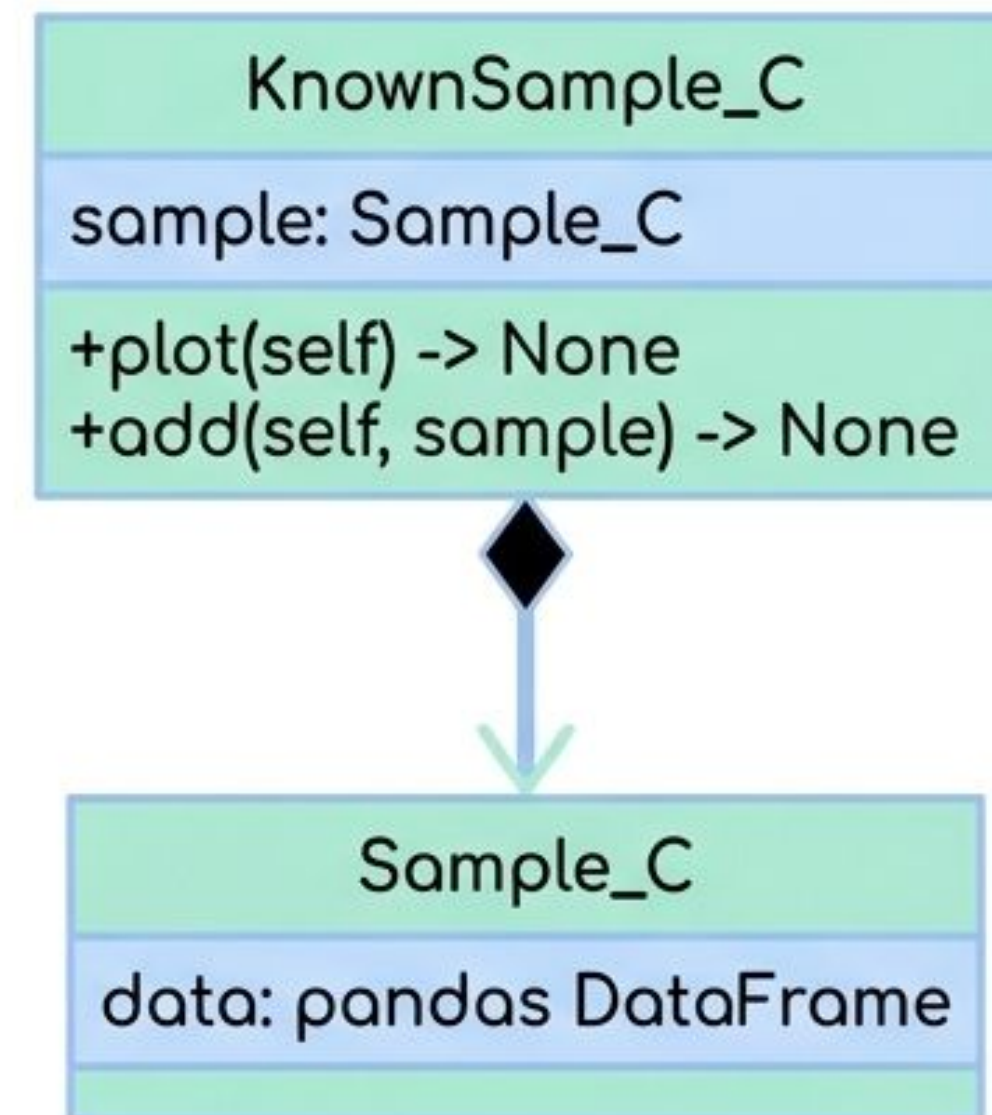
Frente a frente: El diseño estructural

La herencia expande la estructura verticalmente (forzando una relación estática). La composición separa las responsabilidades horizontalmente (permitiendo intercambiar piezas).

Herencia



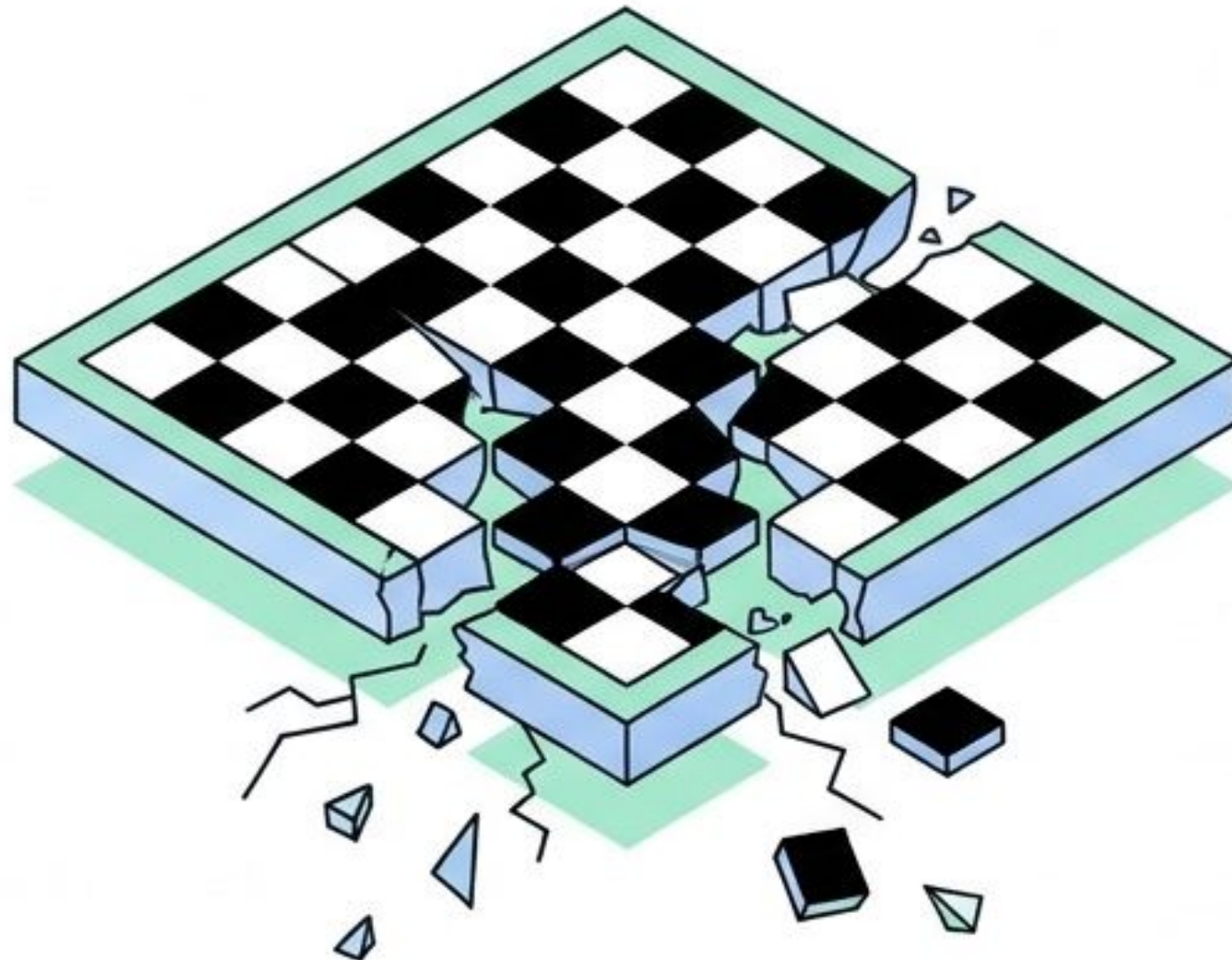
Composición



Matiz técnico: Composición vs Agregación

La composición estricta destruye las partes si el contenedor es destruido. En la agregación, las piezas son inyectadas y pueden sobrevivir de forma independiente.

Composición



Agregación



La Matriz Diagnóstica

Atributo	Herencia	Composición
Relación	'Es-Un' (Subclase)	'Tiene-Un' (Instancia)
Acoplamiento	Fuerte (Caja Blanca / Rígido)	Débil (Caja Negra / Flexible)
Cambios en Ejecución	Imposible (Estático)	Dinámico (Intercambiable)
Reutilización	Comparte código interno	Usa interfaces externas

Prioriza Composición



Este mantra de la ingeniería de software moderna asegura que tu código se mantenga modular, fácil de testear (Unit Testing) y resistente a los cambios futuros.

¿Cuándo usar Herencia?



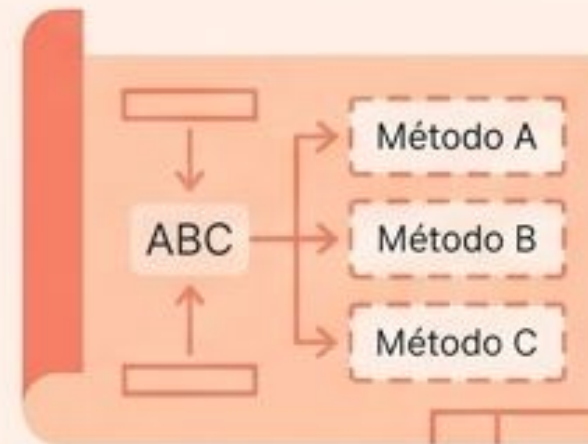
Identidad Pura

La relación "Es-Un" es absoluta e inmutable (ej. Un Cuadrado es un Polígono).



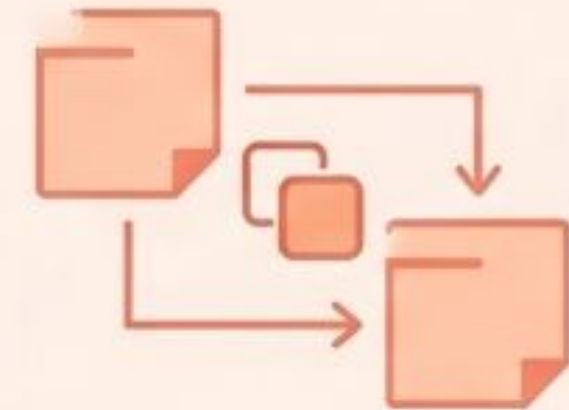
Interfaces Comunes

Necesitas una Clase Base Abstracta (ABC) para garantizar que todas las hijas tengan los mismos métodos.



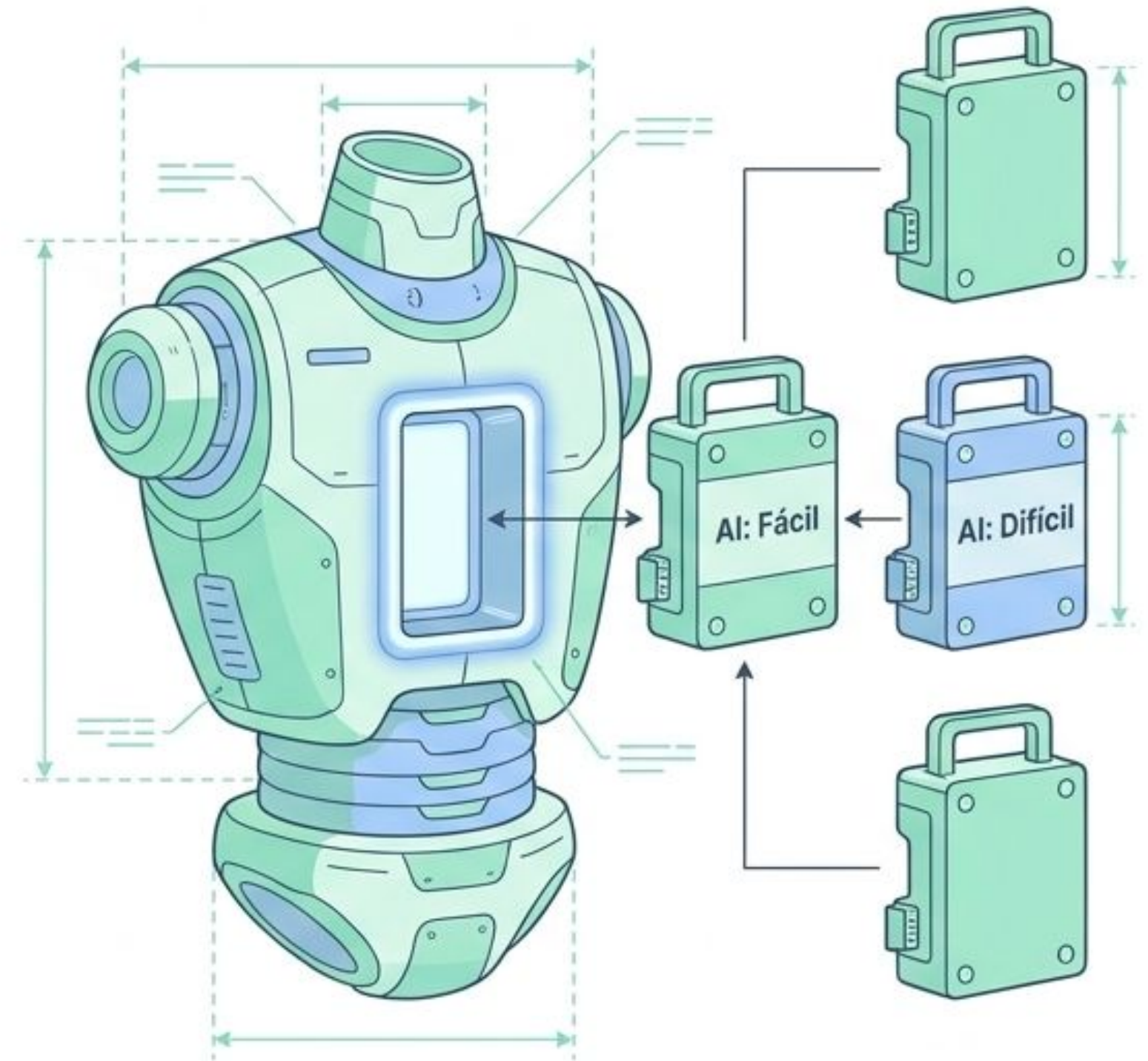
Comportamiento Idéntico

El método de la clase padre se usará exactamente igual en todas las subclases sin excepciones.



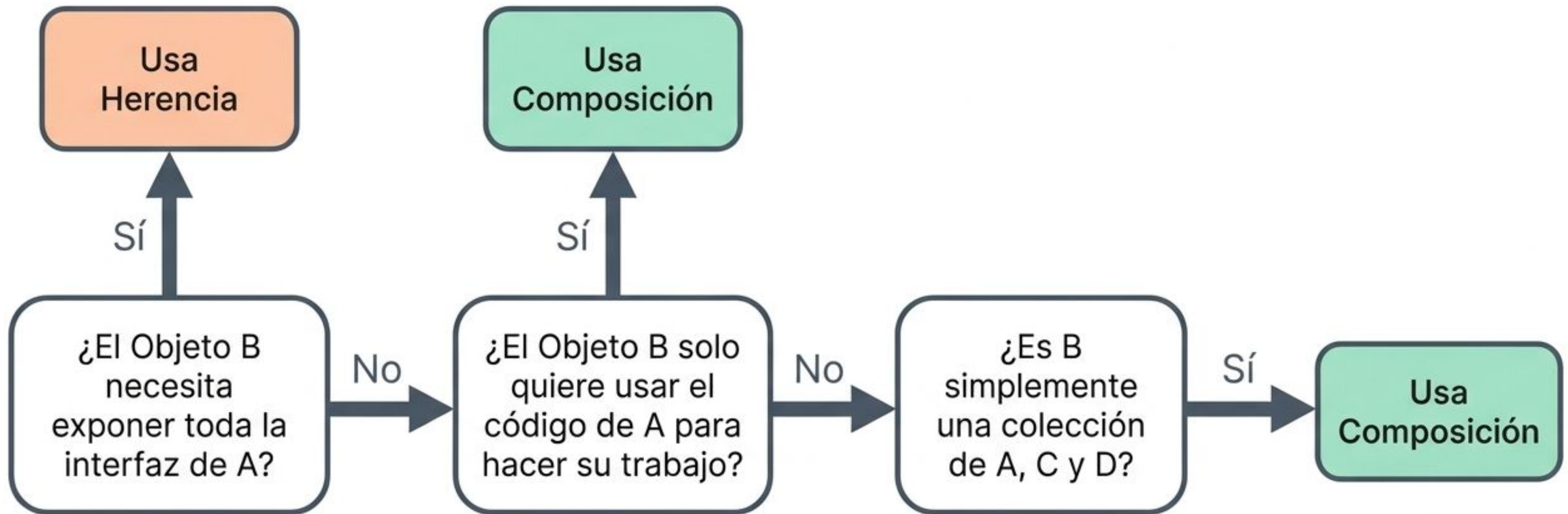
¿Cuándo usar Composición?

- 
Múltiples Partes: El objeto es claramente un ensamblaje de componentes independientes.
- 
Estrategias Dinámicas: Necesitas intercambiar el comportamiento del objeto en tiempo de ejecución (Strategy Pattern).
- 
Testing Aislado: Quieres probar componentes individuales sin cargar todo el árbol de dependencias.



El Árbol de Decisión

Ante la duda, elige piezas modulares. Construye software que sea fácil de ensamblar, y aún más fácil de desmontar.



Composición

```
1 # --- Comportamientos de Movimiento ---
2 class Caminar:
3     def mover(self, nombre):
4         return f"{nombre} avanza caminando por el suelo."
5
6 class Volar:
7     def mover(self, nombre):
8         return f"{nombre} vuela majestuosamente por los cielos."
9
10
11 # --- Comportamientos de Ataque ---
12 class AtaqueEspada:
13     def atacar(self, nombre):
14         return f"{nombre} lanza un tajo feroz con su espada."
15
16 class AtaqueHechizo:
17     def atacar(self, nombre):
18         return f"{nombre} lanza una poderosa bola de fuego."
19
20
21
22
23
24
25
26
27
28
```